

PIM LAB 2 - DPU computing

● DPU Memory management 相關教學筆記

➤ [UPMEM DPU Memory management 相關教學 - HackMD](#)

● Lab requirements

➤ 作業說明

本次作業會讓同學練習使用 DPU 進行 [Hamming distance](#) 的運算。

Hamming distance 是指兩條字串在對應位置字元不同的次數，如下圖：

字串1	a	b	c	d	e	f	g	h
字串2	a	b	a	d	d	c	g	h
Hamming distance	0	0	1	0	1	1	0	0
$0 + 0 + 1 + 0 + 1 + 1 + 0 + 0 = 3$								
兩條字串間同位置有3個字元不相等，因此結果為3。								

請同學先下載本次作業的[程式碼](#)，其中紅色標示為需完成的內容：

configure.h	
dpu_host.c	← TODO! (傳資料給DPU計算、取回結果)
dpu_host.h	
dpu_program.c	← TODO! (宣告WRAM或MRAM、計算hamming distance)
main.c	
makefile	

該程式會由 main.c 開始，會自動產生以下資料(字串長度固定為 8)：

- 1 條輸入字串([input_string](#))
- 32 條待計算的字串([strings](#))

程式的目標是計算 [input_string](#) 與 [strings](#) 中每一條字串之間的

Hamming distance，因此最後會得到 32 筆計算結果。

程式中已提供 CPU 版本的 Hamming distance 計算，作為正確答案，同學僅需完成 DPU 版本的運算流程，也就是填寫：

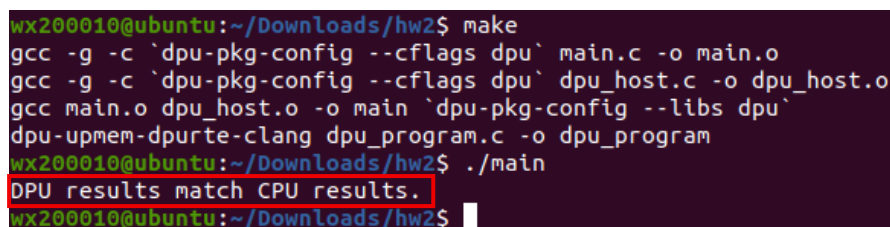
- [dpu_host.c](#) 中的 TODO 部分
- [dpu_program.c](#) 中的 TODO 部分

由於該作業會分配 4 顆 DPU (詳情見 configure.h)，因此同學可將 32 條字串的計算請求分配給 4 顆 DPU。(例：每顆 DPU 僅需計算 8 條字串) 程式執行後，main.c 會去比較 DPU 的計算結果是否與答案一致。

在完成 TODO 後，請依照以下內容錄製成 Demo 影片(不用開鏡頭)：

1. 展示程式執行結果。
2. 解釋你的 DPU 程式碼大概是怎麼設計的。
3. 若將 configure.h 裡的 NUM_STRINGS 從 32 條變為 33 條後，你的程式仍然能順利運行嗎？說明你是怎麼解決的。

➤ 範例結果



```
wx200010@ubuntu:~/Downloads/hw2$ make
gcc -g -c `dpu-pkg-config --cflags dpu` main.c -o main.o
gcc -g -c `dpu-pkg-config --cflags dpu` dpu_host.c -o dpu_host.o
gcc main.o dpu_host.o -o main `dpu-pkg-config --libs dpu`
dpu-upmem-dpurte-clang dpu_program.c -o dpu_program
wx200010@ubuntu:~/Downloads/hw2$ ./main
DPU results match CPU results.
wx200010@ubuntu:~/Downloads/hw2$
```

➤ 評分標準

1. (40%) 影片中有展示程式執行結果，且結果正確(如上方範例結果)
2. (40%) 影片中有解釋自己的程式碼內容
3. (20%) 影片中有將 configure.h 裡的 NUM_STRINGS 修改成 33，且結果仍然正確。

➤ 繳交說明

僅需繳交一份 mp4 影片檔至 moodle 繳交區。

檔名規則為：學號_PIM_lab2.mp4 (例如：P76134260_PIM_lab2.mp4)

若影片無法上傳 moodle，也可先上傳至雲端硬碟，再繳交雲端連結。

➤ 繳交期限

繳交截止日預計為 **12/8 23:59** (以 moodle 繳交區日期為主)

每遲交一天會扣該作業 5 分，超過 1 個禮拜繳交則 0 分計算。